23-Python Modules and File Handling

Introduction

- The lecture covers the creation and usage of modules in Python and introduces comprehensive file handling concepts.

- Live coding, Q&A, and practical examples are used throughout.

Creating and Importing Python Modules

- Any `.py` file is considered a Python module. Code must be placed in `.py` files (not `.ipynb`) for importation.

- Editors like PyCharm (community edition is free), Thonny, and Sublime Text can be used for developing Python files.

- Demo: Folder and file creation, function definitions for addition, checking even numbers, and filtering sequences.

- External libraries (e.g., numpy) may need to be installed in the project environment.

- Functions from one module can be imported and reused in other files for modular coding practices.

Import Methods and Module Namespaces

- Several forms of importing exist:

  - `import module` (full namespace required to access functions).

- `import module as alias` (use alias as namespace).

- `from module import function1, variable2`
(direct import of names, no full module needed).

- `from module import *`
(imports all names; not recommended due to namespace clutter and potential for function overwrites).

• Built-in modules (e.g., math, sys, os, random, datetime) and installed packages (e.g., numpy) differ in origin but are imported similarly.

• Only names in the current namespace after an import can be directly used.

Module Namespaces and __name__ Variable

• Python assigns a special variable `__name__` to every module:

- When a module runs directly, `__name__` is set to `'__main__'`.

- When a module is imported, `__name__` is set to its module name.

• Placing executable code under `if __name__ == '__main__':`
ensures it runs only when the file is executed directly, not on import.

• Prevents unwanted execution of code blocks when importing modules into other files.

• Clarification with code demos and practical imports, both for built-in and user modules.

File Handling in Python

• File handling enables reading and writing data to files, beyond standard input/output.

- Two primary file types:
  - Text files (e.g., `.txt`, `.py`, `.doc`)
  - Binary files (e.g., images, PDFs; data stored in binary format)
- Main use cases: Reading, writing, and appending data.
- Specialized formats (CSV, Excel, JSON) handled typically through libraries (e.g., Pandas), but basic concepts generalize.

File Modes and Operations

- A file must be opened before it can be read or written.
- The `open()` function: `open(filename, mode)` returns a file object.
- Modes:
  - `r`: Read mode (default, cursor at beginning, raises error if file doesn't exist)
  - `w`: Write mode (deletes/recreates file, cursor at beginning, creates file if absent)
  - `a`: Append mode (cursor at end, creates file if absent)
  - `r+`: Read and write
  - `w+`: Write and read (deletes content)
  - `a+`: Append and read

- Add `b` for binary files (e.g., `rb`, `wb`).

File Reading and Writing Methods

- `f.read()`: Reads the entire file, returns a string.
- `f.read(n)`: Reads `n` characters from the current cursor position.
- `f.readline()`: Reads a single line from the current cursor.
- `f.readline(n)`: Reads up to `n` characters from the current line.
- `f.readlines()`: Reads all lines, returns a list of strings.
- `f.write(str)`: Writes a string to the file.
- `f.writelines(list_of_strings)`: Writes a list of strings to the file.
- Each line should generally end with a newline character `\n`.

Cursor Management (tell and seek)

- `f.tell()`: Returns current cursor position in the file (integer offset).
- `f.seek(n)`: Moves the cursor to position `n`.
- File reading/writing starts/moves from the cursor position.

- Read mode: Cursor at start.

- Append mode: Cursor always at end regardless of `seek()`.

- Changes made when file is open persist only after proper closure.

Using the with Statement

- The `with open(...) as f:` construct is recommended for file handling.

- Automatically closes the file after the indented block is executed, even in case of errors.

- Ensures data is saved and resources are released properly without the need for `f.close()`.

Questions & Answers / Classroom Clarifications

- File locations: Files must be in the same directory as the scripts importing them or full path must be provided.

- Handling modules and package imports from other directories is a topic reserved for object-oriented programming/package management classes.

- Clarification on using main blocks, importing modules, managing namespaces, and mutual imports between modules.

- Demonstrated consequences of not properly closing files.

- Best practices highlighted: Use of main guards, explicit closing of files, and with statements for safety.

Conclusion

- Next class topics: Exception handling and regular expressions (regex).

- Assignment: Practice function dictionaries and file handling; submission by 10th April.

- Emphasis on experimenting with file handling methods, understanding modules, and using proper coding practices for scalable Python development.